

Q1. What is Python?

1. **Python is a high-level, interpreted, general-purpose programming language** known for its simple and readable syntax.
2. **It was created by Guido van Rossum** and first released in 1991. Python is developed as an open-source language under the Python Software Foundation.
3. **Python is dynamically typed**, meaning you do not need to explicitly declare a variable's data type. The type is determined at runtime.
4. `x = 10`

`name = "Anshu"`

5. **Python supports multiple programming paradigms**, including procedural, object-oriented, and functional programming.
6. **Python is widely used** in web development, automation, data analysis, artificial intelligence, machine learning, cybersecurity, scripting, and software development. Popular frameworks and libraries include Django, Flask, NumPy, Pandas, and TensorFlow.
7. **Interview point:** Python is interpreted and dynamically typed, but technically Python code is first compiled into **bytecode (.pyc)**, which is then executed by the Python Virtual Machine (PVM). This distinction is important in interviews.

Q2. Why is Python popular?

1. **Simple and readable syntax** — Python uses clean, English-like syntax, making programs easier to write, understand, and maintain.
2. **Easy to learn** — It has a relatively low learning curve compared with languages such as C++ or Java, so beginners can become productive quickly.
3. **Large standard library and ecosystem** — Python provides many built-in modules, while its huge third-party ecosystem includes NumPy, Pandas, Django, Flask, TensorFlow, PyTorch, and Scikit-learn.
4. **Wide range of applications** — Python is used in **web development, automation, data science, AI/ML, cybersecurity, scripting, testing, and DevOps**.
5. **Strong community support** — Python has a very large global developer community, providing extensive documentation, tutorials, open-source projects, and solutions to common problems.
6. **High developer productivity** — Python allows developers to implement functionality with relatively little code.
Interview point: Python's popularity comes from the combination of readability, productivity, ecosystem, community support, and versatility—not simply because it is "easy."

Q3. What are the key features of Python?

1. **Simple and readable** — Python has clean, English-like syntax and uses indentation to define code blocks, making programs easier to read and maintain.
2. **Interpreted and high-level** — Python code is executed by the Python interpreter, and developers do not need to manage low-level memory operations directly.
3. **Dynamically typed** — Variable types are determined at runtime, so explicit type declarations are generally not required.

`x = 10`

`x = "Python"`

4. **Object-oriented and multi-paradigm** — Python supports OOP concepts such as classes, objects, inheritance, polymorphism, and encapsulation, while also supporting procedural and functional programming.

5. **Extensive libraries and frameworks** — Python provides a large standard library and third-party ecosystem for tasks such as web development, data analysis, AI/ML, automation, and networking.
6. **Portable and open source** — Python is free and runs on major platforms such as Windows, Linux, and macOS.
Interview point: Python also provides automatic memory management through garbage collection, reducing the need for manual memory management.

Q4. What are variables in Python?

1. **A variable is a name/reference used to store or refer to a value in Python.** Example:

```
age = 22
name = "Anshu"
```

2. **Python variables do not require explicit type declarations.** The type is determined dynamically at runtime.

```
x = 10    # int
x = "Python" # str
```

3. **Variables are references to objects in memory**, rather than fixed memory boxes containing values. For example, `x = 10` makes `x` refer to the integer object 10.

4. **Python variables can be reassigned and can change their type** during execution because Python is dynamically typed.

```
x = 10
x = 10.5
x = "hello"
```

5. **Variable names must follow Python's naming rules.** They can contain letters, digits, and `_`, cannot start with a digit, and are case-sensitive.

```
student_name = "Anshu" # valid
# 2name = "Anshu"      # invalid
```

6. **Interview point:** Python uses **dynamic typing and strong typing**. Dynamic typing means type checking happens at runtime; strong typing means Python generally does not perform arbitrary implicit conversions between incompatible types, such as `"10" + 5`.

Q5. What are data types in Python?

1. **Data types define the kind of value an object represents** and determine what operations can be performed on that value. Python is **dynamically typed**, so you do not need to declare the type explicitly.

2. **Numeric types** include:

```
int    # 10
float  # 10.5
```

```
complex # 2 + 3j
```

3. **Sequence types** include:

```
str # "Python"
list # [1, 2, 3]
tuple # (1, 2, 3)
range # range(5)
```

4. **Other important built-in types** include:

```
bool # True, False
```

```
set # {1, 2, 3}
```

```
dict # {"name": "Anshu"}
```

None # represents absence of a value

5. **Python also supports user-defined data types** through classes and objects. You can create your own types using:

```
class Student:
```

```
    pass
```

6. **Interview point:** Python is **dynamically typed but strongly typed**, and everything in Python is an **object**, including integers, strings, functions, and classes. You can check an object's type using `type()`:

```
type(10) # <class 'int'>
```

Q6 . What is an integer (int)?

1. **An integer (int) is a built-in Python data type used to represent whole numbers**, without a fractional or decimal part. Examples: 10, 0, -25.
2. **Python integers can be positive, negative, or zero.**
3. **Python integers have arbitrary precision**, meaning they can grow beyond the limits of a fixed 32-bit or 64-bit integer, as long as sufficient memory is available.
4. **Integers support arithmetic operations** such as addition, subtraction, multiplication, floor division, modulo, and exponentiation.
5. **You can convert other compatible values to an integer** using `int()`.
6. **Interview point:** Python's `int` is an **immutable object**. Also, unlike C/C++/Java, Python does not normally have separate `int`, `long`, or `short` types for different integer sizes; `int` handles arbitrary-sized integers.

Q7. What is a float?

1. **A float is a built-in Python data type used to represent numbers with a fractional/decimal component.** Examples include 10.5, 3.14, and -2.75.
2. **Floats can represent positive and negative decimal values, as well as zero.**
3. **Python floats are generally implemented using IEEE 754 double-precision floating-point representation**, typically providing about 15–17 significant decimal digits of precision.
4. **Floats support arithmetic operations** such as addition, subtraction, multiplication, division, and exponentiation.
5. **You can convert values to a float** using `float()`.

```
float(10) # 10.0
```

6. **Interview point:** Floats can have **precision issues** because many decimal values cannot be represented exactly in binary floating-point. For example:

```
0.1 + 0.2
```

may produce 0.30000000000000004. For exact decimal calculations, Python's `Decimal` type is more appropriate.

Q8. What is a string?

1. **A string (str) is a built-in Python data type used to represent a sequence of Unicode characters.** Examples are "Python", 'Hello', and "123".

2. **Strings can be created using single, double, or triple quotes.**
3. **Strings are ordered and support indexing and slicing.** Indexing starts from 0.
4. **Strings are immutable**, meaning their existing characters cannot be changed directly.
5. **Python provides many built-in string operations and methods**, such as `upper()`, `lower()`, `replace()`, `split()`, `strip()`, and `join()`.
6. **Interview point:** Python strings are **Unicode**, so they can represent characters from many languages. They are also iterable and hashable, which allows strings to be used as dictionary keys and set elements.

Q9 . What is a boolean?

1. **A Boolean (bool) is a Python data type that represents one of two logical values:** True or False.
2. **Booleans are commonly used in conditions and decision-making.**
3. **Boolean values are produced by comparison and logical operators.**
4. **Python's bool is a subclass of int.** Therefore:

```
True == 1 # True
```

```
False == 0 # True
```

Internally, True behaves like 1 and False like 0.
5. **Python supports truth-value testing**, where objects can be evaluated as truthy or falsy. For example, 0, "", [], {}, and None are generally falsy, while non-empty values are generally truthy.
6. **Interview point:** Boolean values are essential for **control flow**, including if, while, conditional expressions, and logical operations. The two Boolean singleton objects are True and False.

Q10. What is a list?

1. **A list is an ordered, mutable collection in Python** used to store multiple values in a single variable. It can contain different data types.
2. **Lists are ordered and indexed**, with indexing starting from 0.
3. **Lists are mutable**, meaning elements can be added, removed, or modified after creation.
4. **Lists can contain duplicate values and different data types.**
5. **Python provides many list methods**, including `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `sort()`, and `reverse()`.
6. **Interview point:** Lists are implemented internally as **dynamic arrays**, providing average **O(1) indexing** and amortized **O(1) append**, while insertion or deletion near the beginning/middle is typically **O(n)** because elements may need to be shifted.

Q11. What is a tuple?

1. **A tuple is an ordered, immutable collection in Python** used to store multiple values in a single object.
2. **Tuples are ordered and support indexing and slicing**, just like lists.
3. **Tuples are immutable**, meaning their elements cannot be modified, added, or removed after creation.
4. **Tuples can contain duplicate values and different data types.**
5. **Tuples are useful for fixed collections of related values**, such as coordinates, database records, or returning multiple values from a function.

6. **Interview point:** Tuples are generally more memory-efficient than lists and can be used as **dictionary keys or set elements if all their elements are hashable**. Their immutability also makes them suitable for representing data that should not change.

Q12. What is a set?

1. **A set is an unordered, mutable collection of unique elements in Python.** It automatically removes duplicate values.
2. **Sets do not support indexing or slicing** because they are not sequence types.
3. **Sets are mutable**, so elements can be added or removed using methods such as `add()`, `remove()`, and `discard()`.
4. **Sets support mathematical set operations** such as union, intersection, difference, and symmetric difference.
5. **Sets are commonly used for fast membership testing and removing duplicates.**
6. **Interview point:** Sets are implemented using a **hash table**, so membership testing (`x in set`) is **$O(1)$ average case**, making them much faster than searching through a list, which is **$O(n)$ average case**. Set elements must be **hashable**.

Q13. What is a dictionary?

1. **A dictionary (dict) is a mutable collection that stores data as key-value pairs.** Each key maps to a corresponding value.

```
student = {"name": "Anshu", "age": 22}
```

2. **Keys must be unique and hashable**, while values can be of almost any Python data type.

```
data = {1: "one", 2: "two"}
```

3. **Dictionaries are mutable**, so you can add, update, and delete key-value pairs.

```
student["age"] = 23
student["city"] = "Hyderabad"
del student["city"]
```

4. **You access values using their keys**, rather than numerical indexes.

```
student["name"] # "Anshu"
```

Accessing a missing key directly raises a `KeyError`; `get()` can be used when you want a default instead.

5. **Dictionaries are commonly used for fast lookups, counting, grouping, and representing structured data.**

```
frequency = {}
for x in [1, 2, 2, 3]:
    frequency[x] = frequency.get(x, 0) + 1
```

6. **Interview point:** Dictionaries are implemented using a **hash table**. Lookup, insertion, and deletion are **$O(1)$ average case**, but can degrade to **$O(n)$ worst case**. Since Python 3.7, dictionaries officially preserve **insertion order**.

Q14. What is None in Python?

1. **None is a special Python object that represents the absence of a value or "no value".** It is not the same as 0, False, or an empty string `""`.
2. **The data type of None is NoneType.**
3. **Functions use None when they do not return a meaningful value.**
4. **None is commonly used to initialize variables** when a value will be assigned later.

5. **Use is to check for None, not ==.**

```
if result is None:
```

```
    print("No value")
```

None is a singleton object, so identity checking with is is the correct practice.

6. **Interview point:** None represents **absence of a value**, while values such as 0, False, "", [], and {} are actual objects that happen to be **falsy**. This distinction is important when writing conditions and handling missing values.

Q15. What is type casting?

1. **Type casting is the process of converting a value from one data type to another.** In Python, this is usually done explicitly using built-in functions.
2. **Common type-conversion functions include** int(), float(), str(), bool(), list(), tuple(), and set().

```
x = "100"
```

```
y = int(x)
```

Numeric conversions can change the value.

```
int(10.8) # 10
```

```
float(10) # 10.0
```

int() truncates the fractional part; it does not round the number.

3. **Strings can be converted when their contents are compatible with the target type.**

```
int("25") # 25
```

```
float("3.14") # 3.14
```

```
str(100) # "100"
```

4. **Some conversions are not valid and raise exceptions.**

```
int("Python")
```

```
# ValueError
```

Also, converting between collections can change their properties, such as list(set_values) removing duplicates if the source was a set.

5. **Interview point:** Python mainly uses **explicit type conversion**, but it also performs some **implicit type promotion** in compatible numeric operations.

```
10 + 2.5 # 12.5
```

Here, the integer is promoted to a floating-point value for the operation.

Q16. What is the type() function?

1. **The type() function is a built-in Python function used to determine the data type or class of an object.**

```
x = 10
```

```
print(type(x))
```

```
# <class 'int'>
```

2. **It can be used with different Python data types.**
3. **The function returns a class/type object**, not a string.

4. It is useful for checking the type of a variable during debugging or runtime.
5. `type()` can also be used with three arguments to dynamically create a class, although this is an advanced use.
6. **Interview point:** For checking whether an object belongs to a type, `isinstance()` is generally preferred over `type() ==` because `isinstance()` also considers inheritance.

Q17. What are Python keywords?

1. **Python keywords are reserved words** that have a predefined meaning in the Python language and cannot be used as variable, function, or class names.
2. **Examples of Python keywords** include:

```
if, else, elif, for, while, def, class, return,  
import, from, try, except, True, False, None
```

3. **Keywords are used to define the structure and behavior of Python programs.**

```
if age >= 18:  
    print("Adult")
```

Here, `if` is a keyword.

4. **You cannot use a keyword as an identifier.**
5. **You can view Python's keywords using the keyword module.**

```
import keyword  
  
print(keyword.kwlist)
```

6. **Interview point:** Python has both regular keywords and **soft keywords** in newer versions. Soft keywords such as `match`, `case`, `_`, and `type` are treated as keywords only in specific language contexts, rather than being universally reserved.

Q18. What is an operator?

Definition: An **operator** is a symbol or keyword used to perform a specific operation on one or more operands (values or variables).

1. **Arithmetic Operators:** Used to perform **mathematical calculations**.
Types: `+`, `-`, `*`, `/`, `//`, `%`, `**`
2. **Comparison Operators:** Used to **compare two values** and return `True` or `False`.
Types: `==`, `!=`, `>`, `<`, `>=`, `<=`
3. **Assignment Operators:** Used to **assign values to variables or update their values**.
Types: `=`, `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, `&=`, `|=`, `^=`, `<<=`, `>>=`
4. **Logical Operators:** Used to **combine or negate Boolean conditions**.
Types: `and`, `or`, `not`
5. **Bitwise Operators:** Used to perform operations on the **individual binary bits of integer values**.
Types: `&`, `|`, `^`, `~`, `<<`, `>>`
6. **Membership Operators:** Used to check whether a **value exists inside a sequence or collection**.
Types: `in`, `not in`
7. **Identity Operators:** Used to check whether **two variables refer to the same object in memory**.
Types: `is`, `is not`

8. **Conditional/Ternary Operator:** Used to **select one of two values based on a condition**.

Type: value_if_true if condition else value_if_false

Important: Strictly speaking, Python documentation generally categorizes operators into **7 major operator groups**; the conditional/ternary expression is commonly taught separately as the **8th type** in interview preparation.

Q19. What are if, elif, and else statements?

1. **if statement** — It is used to execute a block of code when a specified condition is **True**.

```
if age >= 18:  
    print("Eligible")
```

2. **elif statement** — It stands for "**else if**" and is used to check another condition when the previous if or elif condition is False.

```
if marks >= 90:  
    print("A")  
  
elif marks >= 75:  
    print("B")
```

3. **else statement** — It executes a block of code when **none of the preceding if or elif conditions are True**.

```
if marks >= 40:  
    print("Pass")  
  
else:  
    print("Fail")
```

4. **Together**, if, elif, and else are used for **decision-making and controlling program flow** based on different conditions.

5. **Execution flow:** Python checks conditions **from top to bottom**. Once one condition is True, its block executes and the remaining elif/else blocks are skipped.

```
if x > 0:  
    print("Positive")  
  
elif x < 0:  
    print("Negative")  
  
else:  
    print("Zero")
```

6. **Interview point:** An if statement is required, while elif and else are optional. You can have **multiple elif blocks**, but only **one final else block**. else does not have a condition of its own.

Q20. What is a Loop in Python?

Definition: A **loop** is a control-flow mechanism used to repeatedly execute a block of code as long as a specified condition is satisfied or for each item in an iterable.

Purpose: Loops are used to avoid writing the same code repeatedly. They are commonly used for **iterating through lists, processing data, repeating calculations, and automating tasks**.

1. **for loop:** A for loop is used to **iterate over the elements of an iterable**, such as a list, tuple, string, dictionary, set, or range.

```
for i in range(5):
```

```
    print(i)
```

Output: 0 1 2 3 4

2. **while loop:** A while loop repeatedly executes a block of code **as long as its condition remains True**.

```
i = 0
```

```
while i < 5:
```

```
    print(i)
```

```
    i += 1
```

3. **Nested loop:** A loop placed **inside another loop** is called a nested loop. The inner loop executes completely for each iteration of the outer loop.

```
for i in range(3):
```

```
    for j in range(2):
```

```
        print(i, j)
```

4. **Interview point:** Python does **not have a built-in do-while loop**. Loop execution can be controlled using break to terminate a loop, continue to skip the current iteration, and pass as a placeholder. for is generally preferred when iterating over a known iterable, while while is useful when repetition depends on a condition.

Q21. What are break and continue statements?

1. **break statement — Definition:** break is a loop-control statement used to **immediately terminate the nearest enclosing loop**, even if the loop condition is still True.
2. **continue statement — Definition:** continue is a loop-control statement used to **skip the remaining statements of the current iteration** and move to the next iteration of the loop.
3. **Main difference:** break **stops the entire loop**, whereas continue **only skips the current iteration** and allows the loop to continue.

break → exit loop completely

continue → skip current iteration

4. **Interview point:** Both statements affect the **nearest enclosing loop**. In nested loops, break or continue applies only to the inner loop where it appears, not all loops. A break can also be used in a while loop to prevent an otherwise infinite loop.

Q22. What is a Function in Python?

1. **Definition:** A **function** is a reusable block of code designed to perform a specific task. A function can accept inputs, process them, and optionally return an output.

```
def add(a, b):
```

```
    return a + b
```

2. **Why are functions used?** Functions provide **code reusability**, so you can write a piece of logic once and call it multiple times instead of repeating the same code.

3. **Functions improve code organization and readability.** Large programs can be divided into smaller, logical functions, making the code easier to understand, test, debug, and maintain.
4. **Built-in functions:** Python provides functions that are already defined and ready to use.

```
print("Hello")
```

5. **User-defined functions:** Functions created by the programmer using the def keyword.

```
def square(n):  
    return n * n  
  
print(square(5)) # 25
```

6. **Main types of functions:** In basic Python classification, functions include **built-in functions**, **user-defined functions**, **anonymous/lambda functions**, **recursive functions**, and **generator functions**. **Interview point:** A function may take **parameters/arguments** and may return a value using return; if no explicit return is executed, the function returns None.

Q23. What are return, Lambda Functions, and Recursion in Python?

1. **return statement:** The return statement is used inside a function to **send a value back to the caller and terminate the function's execution**.

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result) # 8
```

If a function has no return statement, it implicitly returns None.

2. **Lambda function:** A lambda function is a **small anonymous function** created using the lambda keyword. It is generally used when a simple function is needed for a short operation.

```
square = lambda x: x * x  
  
print(square(5)) # 25
```

Syntax:

lambda arguments: expression

3. **Lambda characteristics:** A lambda can take multiple arguments but contains **only one expression**, whose result is automatically returned.

```
add = lambda a, b: a + b
```

Lambda functions are commonly used with map(), filter(), sorted(), etc.

4. **Recursion:** Recursion is a technique where a **function calls itself** to solve a problem by breaking it into smaller versions of the same problem.

```
def countdown(n):  
    if n == 0:  
        return  
  
    print(n)
```

```
countdown(n - 1)
```

5. **Base case and recursive case:** Every recursive function should normally have a **base case** to stop recursion and a **recursive case** that calls the function again.
6. **Interview point:** return transfers control and optionally a value back to the caller; **lambda** is best suited for short, simple operations; **recursion** is useful for problems naturally represented as smaller subproblems, such as tree traversal, DFS, factorial, and divide-and-conquer algorithms. Python does **not optimize tail recursion**, and excessive recursion can raise `RecursionError`.

Q24. What are Module, Package, and PIP in Python?

1. **Module:** A module is a **single Python file (.py)** containing **Python code**, such as functions, classes, and variables, that can be imported and reused in another program.

```
# calculator.py

def add(a, b):

    return a + b
```

You can use it with:

```
import calculator
```

2. **Package:** A package is a **collection of related Python modules organized in a directory**. It helps structure large applications into logical components.

```
mypackage/

    __init__.py
    calculator.py
    converter.py
```

You can import a module from it:

```
from mypackage import calculator
```

3. **PIP:** PIP is Python's standard **package installer and package-management tool**. It is used to install, upgrade, and remove packages from the Python Package Index (PyPI) and other package sources.

```
pip install requests
```

4. **Module vs Package:** A **module** is normally one **.py** file, whereas a **package** is a **directory containing related modules/subpackages**.

```
module → calculator.py

package → mypackage/
          |— calculator.py
          └— converter.py
```

5. **Packages installed using PIP:** Developers use PIP to install third-party libraries such as:

```
pip install numpy
```

```
pip install pandas
```

```
pip install requests
```

These packages provide functionality that is not necessarily included in Python's standard library.

6. **Interview point:** Do not confuse **PIP** with **PyPI**. **PIP** is the tool used to install packages, while **PyPI (Python Package Index)** is a repository/index that hosts Python packages. For example:

PyPI → stores/distributes packages

PIP → installs/manages those packages

Q25. What is an Exception and Exception Handling in Python?

1. **Exception:** An exception is an **error or unexpected event that occurs during program execution** and interrupts the normal flow of the program.

```
x = 10 / 0
```

This produces a ZeroDivisionError.

2. **Exception handling:** Exception handling is a mechanism used to **detect, handle, and recover from runtime errors** without abruptly terminating the program. Python mainly uses try, except, else, and finally.

3. **try and except:** The try block contains code that might cause an exception, while except handles the exception.

```
try:
```

```
    x = 10 / 0
```

```
except ZeroDivisionError:
```

```
    print("Cannot divide by zero")
```

4. **else:** The else block executes **only when no exception occurs** in the try block.

```
try:
```

```
    result = 10 / 2
```

```
except ZeroDivisionError:
```

```
    print("Error")
```

```
else:
```

```
    print(result)
```

5. **finally:** The finally block **always executes**, whether an exception occurs or not. It is commonly used for cleanup operations such as closing files or database connections.

```
try:
```

```
    file = open("data.txt")
```

```
except FileNotFoundError:
```

```
    print("File not found")
```

```
finally:
```

```
    print("Execution completed")
```

6. **Interview point:** Python also allows you to **raise exceptions explicitly** using `raise`, and you can create custom exception classes. Common built-in exceptions include `ValueError`, `TypeError`, `IndexError`, `KeyError`, `FileNotFoundError`, and `ZeroDivisionError`.

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

Flow: try → exception? → except → finally; if no exception occurs, try → else → finally.

Q26. What are try, except, else, and finally blocks?

1. **try block — Used for risky code**

The try block contains the code that **may raise an exception**. Python first executes the statements inside this block.

```
try:
    result = 10 / 2
```

2. **except block — Used for handling exceptions**

The except block executes when a specified exception occurs inside the try block. It prevents the program from terminating unexpectedly.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

3. **else block — Used when no exception occurs**

The else block executes **only if the try block completes successfully without raising an exception**.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error")
else:
    print("Result:", result)
```

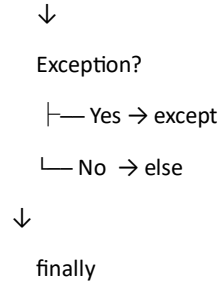
4. **finally block — Used for cleanup**

The finally block executes **whether an exception occurs or not**. It is commonly used for cleanup operations such as closing files, releasing resources, or closing database connections.

```
try:
    file = open("data.txt")
finally:
    file.close()
```

5. **Execution flow:** The four blocks work together like this:

```
try
```



finally normally executes in both cases.

6. **Interview point:** try and except are the core of exception handling. else is optional and should contain code that should run only after a successful try; finally is also optional and is primarily used for **guaranteed cleanup**. You can have multiple except blocks for different exception types.

Q27. What is a decorator?

1. **Definition:** A decorator is a function that **modifies or extends the behavior of another function or class without directly changing its source code**.
2. **Purpose:** Decorators are used for adding **reusable functionality** such as logging, authentication, authorization, timing, caching, and validation.
3. **How it works:** A decorator takes a function as input, creates a wrapper function containing additional behavior, and returns the wrapper.

```
def decorator(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper
```

4. **Using @ syntax:** Python provides a convenient syntax for applying decorators.

```
@decorator
def greet():
    print("Hello")
```

This is essentially equivalent to:

```
greet = decorator(greet)
```

5. **Example:** A decorator can add logging before executing a function.

```
def log(func):
    def wrapper():
        print("Function called")
        func()
    return wrapper
```

```
@log
def hello():
    print("Hello")

hello()
```

6. **Interview point:** Decorators rely on Python's **first-class functions and closures**. In production code, `functools.wraps` should usually be used on the wrapper to preserve the decorated function's metadata such as its name and docstring. Multiple decorators can also be stacked on the same function.

Q28. What is a Generator in Python?

1. **Definition:** A generator is a special type of function that **produces values one at a time instead of generating and storing all values in memory at once**.
2. **Purpose:** Generators are mainly used for **memory-efficient processing of large datasets or sequences**, especially when all values are not needed simultaneously.
3. **yield keyword:** A generator function uses `yield` instead of `return` to produce a value. When `yield` is reached, the function **pauses its execution and preserves its state**.

```
def numbers():
    yield 1
    yield 2
    yield 3
```

4. **Execution:** When you call a generator function, it returns a **generator iterator**; the function body does not execute completely immediately. Each `next()` resumes execution from where it previously paused.

```
g = numbers()
print(next(g)) # 1
print(next(g)) # 2
print(next(g)) # 3
```

5. **Memory advantage:** A normal list stores all values:

```
nums = [x for x in range(1000000)]
```

A generator produces them lazily:

```
nums = (x for x in range(1000000))
```

Therefore, generators are much more memory-efficient for large or potentially infinite sequences.

6. **Interview point:** Generators implement the **iterator protocol** and use lazy evaluation. Once a generator is exhausted, it cannot normally be restarted; you create a new generator if you need to iterate again. They are commonly used for **file processing, streaming data, pipelines, and large datasets**.

Q29. What is a Virtual Environment in Python?

1. **Definition:** A virtual environment is an **isolated Python environment** created for a specific project. It has its own Python interpreter and installed packages, separate from other projects and the system Python environment.

2. **Why is it used?** It prevents **dependency conflicts** between projects. For example, one project may require Django 4, while another requires Django 5.
3. **How to create one:** Python provides the built-in venv module.

```
python -m venv myenv
```

This creates a virtual environment named myenv.

4. **How to activate it:**

On Windows:

```
myenv\Scripts\activate
```

On Linux/macOS:

```
source myenv/bin/activate
```

5. **Package management:** Once activated, packages installed using pip are installed into that environment rather than the global Python environment.

```
pip install pandas
```

You can record dependencies using:

```
pip freeze > requirements.txt
```

6. **Interview point:** Virtual environments provide **project-level dependency isolation and reproducibility**. They are not containers or virtual machines; they isolate Python packages and the Python environment at the project level, while sharing the underlying operating system and hardware.